Spécification des Exigences Logicielle

Table des Matières

1. Introduction
    - 1.1 But
    - 1.2 Périmètre
    - 1.3 Définitions, Acronymes et Abréviations
    - 1.4 Conventions du Document
    - 1.5 Références

2. Description Générale
    - 2.1 Perspective du Produit
    - 2.2 Classes et Caractéristiques des Utilisateurs
    - 2.3 Environnement d'Exploitation
    - 2.4 Contraintes de Conception et de Mise en Œuvre
    - 2.5 Hypothèses et Dépendances

3. Architecture Système et Conception Technique
    - 3.1 Style Architectural
    - 3.2 Conception Technique par Composant

4. Exigences Fonctionnelles Détaillées
    - 4.1 Module de Gestion de Compte et de Vérification
    - 4.2 Module de Gestion de l'Infrastructure de Stationnement
    - 4.3 Module de Découverte du Stationnement
    - 4.4 Module de Réservation (Booking)
    - 4.5 Module d'Abonnement (Subscription)
    - 4.6 Module de Traitement des Paiements
    - 4.7 Module de Contrôle d'Accès par QR Code
    - 4.8 Module d'Occupation en Temps Réel
    - 4.9 Modules Administratifs
    - 4.10 Module d'Audit et de Journalisation

5. Exigences de Conception de la Base de Données
    - 5.1 Pile Technologique de la Base de Données
    - 5.2 Description du Modèle de Données (ERD)
    - 5.3 Intégrité et Règles des Données
    - 5.4 Stratégie d'Indexation

6. Exigences d'Interfaces Externes
    - 6.1 Interfaces Utilisateur (Mobile & Web)

- 6.2 Interfaces Matérielles
- 6.3 Interfaces Logicielles (Intégrations Tierces)
- 6.4 Interfaces de Communication

7. Exigences Non-Fonctionnelles

- 7.1 Exigences de Sécurité
- 7.2 Exigences de Conformité
- 7.3 Exigences de Performance
- 7.4 Disponibilité et Reprise Après Sinistre (PRA)
- 7.5 Exigences de Montée en Charge (Scalabilité)
- 7.6 Auditabilité et Journalisation
- 7.7 Exigences d'Utilisabilité

8. Annexes

- 8.1 Glossaire des Termes
- 8.2 Registre des Risques et Stratégies d'Atténuation
- 8.3 Améliorations Futures (Hors Périmètre v1.0)

---

1. Introduction

1.1 But

Le présent document définit les exigences fonctionnelles et non-fonctionnelles pour le Système de Gestion de Stationnement Intelligent – Alger. Le système est une application mobile et web qui permet aux utilisateurs de localiser, réserver, s'abonner et payer des places de stationnement au sein de parcs de stationnement privés situés à Alger. Ce document constitue la référence principale pour toutes les parties prenantes du projet, y compris l'équipe de développement, l'assurance qualité (QA), les auditeurs de sécurité et les ingénieurs infrastructure, garantissant une compréhension commune et partagée du système à construire.

1.2 Périmètre

Le système fournit un écosystème numérique complet pour la gestion du stationnement, incluant :

- L'enregistrement des utilisateurs, la vérification d'identité et la gestion de documents.
- La découverte de parkings via une carte interactive et le redirection vers un service de navigation.
- Des panneaux d'administration basés sur les rôles (RBAC) pour gérer l'infrastructure, les utilisateurs et les finances.
- Une gestion granulaire des parkings, des étages et des places individuelles.
- La gestion de réservations (courte durée) et d'abonnements (longue durée).
- Un traitement sécurisé des paiements électroniques.
- Une validation d'accès aux parkings par QR code.
- Une surveillance en temps réel de l'occupation des places.
- Une journalisation d'audit complète, alignée sur les contrôles de la norme ISO 27001.

1.3 Définitions, Acronymes et Abréviations

Terme	Définition
RBAC	Contrôle d'Accès Basé sur les Rôles (Role-Based Access Control)
QR Code	Code à Réponse Rapide (Quick Response Code)
CRUD	Créer, Lire, Mettre à Jour, Supprimer (Create, Read, Update, Delete)
ISO 27001	Norme Internationale pour le Management de la Sécurité de l'Information
Carte Grise	Certificat d'immatriculation du véhicule

Terme	Définition
JWT	Jeton Web JSON (JSON Web Token) - pour l'authentification
MFA	Authentification Multi-Facteurs (Multi-Factor Authentication)
TLS	Sécurité de la Couche de Transport (Transport Layer Security) - protocole de chiffrement
AES	Norme de Chiffrement Avancée (Advanced Encryption Standard)
RTO	Objectif de Temps de Reprise (Recovery Time Objective)
RPO	Objectif de Point de Reprise (Recovery Point Objective)
SIEM	Gestion des Informations et des Événements de Sécurité (Security Information and Event Management)
WAF	Pare-feu pour Applications Web (Web Application Firewall)

1.4 Conventions du Document

- FR-X : Identifiant d'une Exigence Fonctionnelle (Fonctionnalité).
- NFR-X : Identifiant d'une Exigence Non-Fonctionnelle (Performance, Sécurité...).
- Encadré: Utilisé pour les exemples techniques, comme les structures de données (payloads).
- Gras : Met en évidence les termes clés et les concepts importants.

1.5 Références

- Norme ISO/CEI 27001:2022 - Sécurité de l'information, cybersécurité et protection de la vie privée.
- Loi Algérienne 18-07 relative à la protection des personnes physiques dans le traitement des données à caractère personnel.

2. Description Générale

2.1 Perspective du Produit

Le Système de Gestion de Stationnement Intelligent est une nouvelle solution, développée sur mesure. Elle remplace les processus de gestion manuels ou inexistants par un système numérique et automatisé. Le système est composé de :

- Application Mobile (Client) : Pour les utilisateurs finaux (gestion de compte, recherche, réservation).
- Panneau d'Administration Web : Pour les opérateurs, agents de vérification et administrateurs.
- Services Backend (API) : Un ensemble de services modulaires gérant toute la logique métier.

- Passerelle de Paiement : Une intégration sécurisée avec un prestataire de paiement tiers.
- Service de Validation QR : Un service dédié pour la validation sécurisée des QR codes.
- Base de Données Centralisée : Pour le stockage persistant des données.
- Système de Journalisation et de Surveillance : Pour la sécurité, l'audit et la supervision opérationnelle.

2.2 Classes et Caractéristiques des Utilisateurs

Classe	Caractéristiques
2.2.1 Client (Utilisateur Final)	• S'enregistre et gère son profil. • Télécharge ses documents d'identité et du véhicule pour vérification. • Découvre les parkings, effectue des réservations et achète des abonnements. • Utilise le QR code généré pour entrer et sortir. • Niveau Technique : Grand public ; l'interface doit être très intuitive.
2.2.2 Opérateur de Parking	• Scanne les QR codes des utilisateurs via l'application mobile. • Surveille l'état en temps réel du parking qui lui est assigné. • Met à jour manuellement le statut des places (ex: pour maintenance). • Niveau Technique : Basique ; nécessite une interface simple et ciblée.
2.2.3 Agent de Vérification	• Examine les permis de conduire et les cartes grises téléchargés. • Approuve ou rejette les comptes utilisateurs en se basant sur la validité des documents. • Niveau Technique : Modéré ; nécessite des outils pour une revue efficace des documents.
2.2.4 Administrateur Infrastructure	• Gère la hiérarchie physique : parkings, étages et places. • Configure les modèles de tarification pour les différents types d'étages et durées. • Niveau Technique : Élevé ; nécessite une interface claire et puissante pour la saisie de données complexes.
2.2.5 Administrateur Financier	• Consulte l'historique des transactions et les statuts de paiement. • Gère les demandes de remboursement et génère des rapports financiers. • Niveau Technique : Modéré ; nécessite un accès à des données financières précises et filtrables.

Classe	Caractéristiques
2.2.6 Super Administrateur	• Possède un accès complet et sans restriction à toutes les fonctions du système. • Crée et gère les autres comptes administrateurs. • Accède aux journaux d'audit complets et immuables. • Effectue des actions irréversibles comme la suppression d'infrastructures. • Niveau Technique : Expert ; nécessite un tableau de bord complet avec un contrôle total.

2.3 Environnement d'Exploitation

• Plateformes Mobiles : Android (API niveau 30+) et iOS (version 14+).
• Navigateurs Web : Dernières versions de Google Chrome, Mozilla Firefox et Microsoft Edge.
• Infrastructure Backend : Hébergée dans le cloud (fournisseur de cloud à Alger) sur des serveurs Linux.
• Communication : Toute communication client-serveur est chiffrée via HTTPS (TLS 1.3).

2.4 Contraintes de Conception et de Mise en Œuvre

• Le système doit être conforme à la réglementation Algérienne (Loi 18-07) concernant le stockage et le traitement des documents personnels.
• La version initiale doit prendre en charge le français, l'anglais et l'arabe.
• L'intégration de la passerelle de paiement est soumise aux capacités et API des partenaires bancaires locaux (ex: SATIM).

2.5 Hypothèses et Dépendances

• Hypothèse : Les utilisateurs finaux possèdent des smartphones avec appareil photo et connexion internet.
• Hypothèse : Les opérateurs de parking disposeront d'appareils fournis par la société pour le scan des QR codes.
• Dépendance : La fonctionnalité de paiement dépend de la disponibilité et de la fiabilité de la passerelle de paiement tierce choisie.
• Dépendance : La fonctionnalité de navigation dépend de la disponibilité et de la précision de Google Maps dans la région d'Alger.

3. Architecture Système et Conception Technique

3.1 Style Architectural

Le système suivra une **Architecture Microservices** modulaire. Cette approche favorise la séparation des préoccupations, la scalabilité indépendante et la maintenabilité. Les services communiqueront via des API RESTful synchrones et, si nécessaire, des files d'attente de messages asynchrones.

Services Principaux :

Service
1. Service d'Authentification
2. Service de Gestion des Utilisateurs
3. Service d'Infrastructure de Stationnement
4. Service de Réservation
5. Service d'Abonnement
6. Service de Paiement
7. Service QR
8. Service de Notification (Email/SMS/Push)
9. Service d'Audit et de Journalisation

3.2 Conception Technique par Composant

3.2.1 Passerelle API (API Gateway)

• Rôles : Point d'entrée unique pour toutes les requêtes client. Gère la terminaison TLS, la limitation de débit (rate limiting), la validation JWT, le routage des requêtes.

• Contrôles de Sécurité : Impose TLS 1.3, s'intègre à un WAF, applique des limites par adresse IP et valide les signatures JWT avant de router les requêtes.

3.2.2 Service d'Authentification

• Stratégie de Jeton : Émet des jetons JWT à courte durée de vie (15 min) et des "refresh tokens" (7-30 jours). Les jetons contiennent l'ID utilisateur, le rôle et une version de jeton.

• Sécurité : Mots de passe hachés avec Argon2id. Authentification Multi-Facteurs (MFA) obligatoire pour tous les rôles administratifs. Blocage de compte après 5 tentatives de connexion échouées.

3.2.3 Service Utilisateur et Vérification

• Gestion des Documents : Les images téléchargées sont chiffrées avec AES-256 et stockées dans un stockage d'objets dédié. Un hash SHA-256 du fichier est stocké en base pour vérification d'intégrité.

• Workflow de Vérification : Gère une machine à états pour le statut utilisateur : EN_ATTENTE_DOCUMENTS → EN_COURS_DE_REVUE → APPROUVE/REJETE. Les transitions sont immuables et journalisées.

3.2.4 Service d'Infrastructure de Stationnement

• Hiérarchie des Données : Gère la relation Parking → Étage → PlaceParking.

• Machine à États des Places : Gère le cycle de vie d'une place : DISPONIBLE ↔ RÉSERVÉE ↔ OCCUPÉE ↔ MAINTENANCE. Les transitions sont déclenchées par les réservations, les paiements et les actions des opérateurs.

3.2.5 Service de Réservation (Booking)

• Logique Métier : Gère la création, la tarification et la gestion des réservations limitées dans le temps. Applique les règles pour empêcher la double réservation.

• Cycle de Vie : CRÉÉE → PAIEMENT_EN_ATTENTE → CONFIRMÉE → ACTIVE → TERMINÉE | EXPIRÉE | ANNULÉE.

3.2.6 Service d'Abonnement (Subscription)

• Logique Métier : Gère les assignations de places à long terme. Une place est exclusivement verrouillée pour un utilisateur pendant la durée de l'abonnement.

• Règles Métier : Le renouvellement automatique n'est autorisé que si le paiement réussit et qu'aucune infraction n'est associée au compte de l'utilisateur.

3.2.7 Service de Paiement

• Flux : Intègre une passerelle de paiement tierce. Génère une session de paiement, gère de manière sécurisée le rappel (webhook) de la passerelle, valide la signature et met à jour le statut de la transaction.

• Idempotence : Toutes les opérations de paiement doivent être idempotentes pour éviter les doubles traitements.

• Statuts : INITIÉ → AUTORISÉ → CAPTURÉ → ÉCHOUÉ → REMBOURSÉ.

3.2.8 Service QR

• Conception du Jeton : Le QR code contient un identifiant de jeton chiffré, une signature numérique (RSA-2048/ECC) et une date d'expiration. Aucune donnée brute (ID utilisateur, ID place) n'est directement exposée.

• Logique de Validation : L'application de scan envoie le jeton au backend. Le backend valide la signature, vérifie l'expiration et confirme le jeton par rapport à la réservation/abonnement actif en base de données.

3.2.9 Service de Notification

• Fonction : Service centralisé pour gérer toutes les communications sortantes : liens de vérification par email, confirmations de paiement par SMS, notifications push pour les rappels de réservation.

3.2.10 Service d'Audit et de Journalisation

• Fonction : Service dédié, en "écriture seule", qui reçoit et stocke les journaux d'audit. Ce service garantit l'immutabilité des journaux et fournit une interface sécurisée pour la consultation par le Super Administrateur.

4. Exigences Fonctionnelles Détaillées

4.1 Module de Gestion de Compte et de Vérification

ID	Description
EX-FR-01 (Enregistrement)	Les utilisateurs s'enregistrent avec email/mot de passe. Politique de mot de passe fort (min 8 caractères, maj/min, chiffre, symbole). Vérification d'email obligatoire.
EX-FR-02 (Vérification d'Identité)	Les utilisateurs téléchargent des images de leur Permis de Conduire et de leur Carte Grise. Le système les stocke de manière chiffrée. Un Agent de Vérification peut approuver/rejeter avec commentaires. Le compte est restreint (EN_ATTENTE_DOCUMENTS, EN_COURS_DE_REVUE) jusqu'à approbation.

4.2 Module de Gestion de l'Infrastructure de Stationnement

ID	Description
EX-FR-03 (Gestion des Parkings)	L'Administrateur Infrastructure peut créer, lire et modifier les parkings (Nom, GPS, adresse, capacité, étages). Seul le Super Admin peut supprimer.
EX-FR-04 (Gestion des Étages)	Les admins définissent les étages, leur type (Journalier, Réservation, Abonnement) et configurent les règles de tarification spécifiques à l'étage.
EX-FR-05 (Gestion des Places)	Les admins définissent les places (numéro, type) sur un étage. Statuts : DISPONIBLE, RÉSERVÉE, OCCUPÉE, MAINTENANCE.

4.3 Module de Découverte du Stationnement

ID	Description
EX-FR-06 (Affichage Carte)	L'app mobile affiche une carte (SDK Google Maps) d'Alger avec des marqueurs personnalisés pour tous les parkings. Un clic sur un marqueur affiche une fiche avec le nom,

ID	Description
	l'adresse et un résumé de disponibilité (ex: "45 places libres").
EX-FR-07 (Navigation)	Un bouton "Naviguer" sur l'écran de détail du parking redirige vers l'application Google Maps avec les coordonnées GPS du parking pré-remplies.

4.4 Module de Réservation (Booking)

ID	Description
EX-FR-08 (Processus de Réservation)	L'utilisateur sélectionne un parking, puis un Étage de Réservation. Le système affiche une grille dynamique des places, colorées par statut. L'utilisateur choisit une place DISPONIBLE et une durée parmi des options configurables (4h, demi-journée, 24h).
EX-FR-09 (Confirmation de Réservation)	Après paiement réussi, le système : (1) Crée un enregistrement de réservation (CONFIRMÉE). (2) Marque la place comme RÉSERVÉE. (3) Déclenche la génération du QR code. (4) Envoie un reçu numérique par email/notification.

4.5 Module d'Abonnement (Subscription)

ID	Description
EX-FR-10 (Processus d'Abonnement)	L'utilisateur sélectionne un parking, puis un Étage d'Abonnement. Le système affiche les places disponibles. L'utilisateur choisit une place et une formule (mensuelle/annuelle). Le paiement est traité. En cas de succès, la place est marquée RÉSERVÉE pour l'utilisateur pour toute la durée, et un QR code persistant est généré.

4.6 Module de Traitement des Paiements

ID	Description
EX-FR-11 (Intégration Paiement)	Le système s'intègre à une passerelle de paiement locale. Le flux de paiement est conforme PCI-DSS via redirection vers la page hébergée par la passerelle. Le backend valide la confirmation asynchrone via un webhook signé. Toutes les transactions sont stockées avec un ID unique.

4.7 Module de Contrôle d'Accès par QR Code

ID	Description
EX-FR-12 (Génération de QR)	Dès qu'une réservation/abonnement est confirmé, le service QR génère un jeton sécurisé, le signe et retourne une image QR à l'application client. Le QR inclut une expiration (heure de fin de réservation ou date de fin d'abonnement).
EX-FR-13 (Validation de QR)	L'Opérateur scanne le QR. L'application envoie le jeton au backend. Le système valide la signature, l'expiration et vérifie que le jeton correspond à une réservation/abonnement actif pour cette place/parking. En cas de succès, l'entrée est autorisée et le statut de la place passe à OCCUPÉE. Tous les scans sont journalisés.

4.8 Module d'Occupation en Temps Réel

ID	Description
EX-FR-14 (Mise à Jour Dynamique de l'UI)	Les interfaces mobile et admin doivent refléter les changements de statut des places en temps réel.
Mise en Œuvre Technique	Une connexion WebSocket sera établie après connexion. Le serveur envoie les mises à jour de statut pour les places de l'étage actuellement visualisé. Un repli sur un polling périodique (toutes les 10 secondes) sera implémenté.

4.9 Modules Administratifs

ID	Description
EX-FR-15 (Panneau Opérateur)	Interface simple listant les places de son parking. Boutons pour changer le statut des places (ex: OCCUPÉE → DISPONIBLE) et un scanner QR.
EX-FR-16 (Panneau Agent de Vérification)	Liste des utilisateurs en attente de vérification. Possibilité de voir les images téléchargées et d'approuver/rejeter.
EX-FR-17 (Panneau Admin Infrastructure)	Interface complexe avec formulaires pour créer/modifier parkings, étages et places. Inclut un éditeur de plan d'étage visuel.
EX-FR-18 (Panneau Admin Financier)	Tableau de bord avec liste des transactions, outils de recherche/filtre, et fonctionnalités pour initier des remboursements et générer des rapports financiers de base (CSV/Excel).
EX-FR-19 (Panneau Super Admin)	Toutes les capacités des admins, plus la gestion des utilisateurs pour les comptes admin et l'accès au visualisateur de journaux d'audit complets et inaltérables.

4.10 Module d'Audit et de Journalisation

ID	Description
EX-FR-20 (Journalisation CRUD)	Toutes les opérations de création, modification et suppression sur les entités critiques (utilisateurs, réservations, paiements, infrastructure) sont journalisées.
EX-FR-21 (Journalisation des Accès)	Les tentatives de connexion réussies et échouées, en particulier pour les rôles admin, sont journalisées.
EX-FR-22 (Détails des Journaux)	Chaque entrée de journal inclut : horodatage, ID de l'acteur, rôle de l'acteur, type d'action, type d'entité, ID de l'entité, adresse IP, et (pour les modifications) les valeurs avant/après au format JSON.

5. Exigences de Conception de la Base de Données

5.1 Pile Technologique de la Base de Données

- Base de Données Principale : PostgreSQL 15+ (Relationnelle, conforme ACID).
- Couche de Cache : Redis 7+ (pour l'occupation en temps réel, gestion de sessions, limitation de débit).
- Stockage d'Objets : AWS S3 / Azure Blob Storage (pour le stockage chiffré des documents et images).

5.2 Description du Modèle de Données (ERD)

La base de données sera conçue avec les entités et relations clés suivantes :

- Utilisateurs (1) -- (0..*) Documents
- Utilisateurs (1) -- (0..*) Réservations
- Utilisateurs (1) -- (0..*) Abonnements
- Parkings (1) -- (0..*) Étages
- Étages (1) -- (0..*) PlacesParking
- PlacesParking (1) -- (0..1) Réservations (Une place peut avoir une réservation active)
- PlacesParking (1) -- (0..1) Abonnements (Une place peut avoir un abonnement actif)
- Réservations (1) -- (1) Transactions
- Abonnements (1) -- (0..*) Transactions
- ValidationsQR (0..*) -- (1) Réservations/Abonnements (Association polymorphe)
- JournauxAudit (0..*) -- (1) Utilisateurs (L'acteur)

5.3 Intégrité et Règles des Données

- Clés Étrangères : Imposées au niveau de la base de données pour toutes les relations (ex: utilisateur_id dans réservations doit référencer un utilisateur_id valide dans utilisateurs).

• Contraintes d'Unicité : Imposées pour éviter la double réservation. Une contrainte unique composite sur (place_id, heure_debut, heure_fin) où le statut est CONFIRMÉE ou ACTIVE.

• Suppressions Logiques (Soft Deletes) : Implémentées pour Utilisateurs, Parkings, et Étages via un booléen est_supprime. La suppression physique est restreinte et uniquement effectuée par le Super Admin dans des circonstances spécifiques et journalisées.

• Immutabilité des Journaux d'Audit : Les enregistrements dans la table journaux_audit sont en "ajout seul". Aucune opération UPDATE ou DELETE n'est permise par l'application.

5.4 Stratégie d'Indexation

Des index seront créés sur toutes les clés étrangères et sur les colonnes fréquemment utilisées dans les clauses WHERE pour la recherche et le filtrage. Des index spécifiques seront également ajoutés pour optimiser les requêtes sur les plages horaires.

6. Exigences d'Interfaces Externes

6.1 Interfaces Utilisateur (Mobile & Web)

• Application Mobile (Flutter/React Native) : Doit être intuitive, réactive et supporter les dispositions RTL/LTR pour l'arabe/français. Doit inclure un scanner de QR code avec accès à la caméra.

• Application Web Admin (React/Angular) : Doit être réactive, riche en fonctionnalités et fournir des interfaces de gestion de données complexes (tableaux, formulaires, filtres). Doit supporter les vues basées sur les rôles.

6.2 Interfaces Matérielles

• Caméra : L'application mobile nécessite l'accès à la caméra du dispositif pour le scan des QR codes.

• GPS : L'application mobile nécessite l'accès à la localisation pour la fonctionnalité de carte.

6.3 Interfaces Logicielles (Intégrations Tierces)

• SDK Google Maps : Pour l'affichage de la carte et la redirection vers l'application de navigation.

• API de la Passerelle de Paiement : Pour le traitement des paiements. L'intégration sera côté serveur, utilisant le SDK ou l'API REST de la passerelle, et gérera les webhooks pour les confirmations de paiement.

• API d'une Passerelle SMS : Pour l'envoi de mots de passe à usage unique (OTP) ou d'alertes critiques (optionnel).

• Service d'Email (SMTP/SendGrid) : Pour l'envoi des emails de vérification, reçus et notifications.

6.4 Interfaces de Communication

• Client-Serveur : Toute communication via HTTPS (TLS 1.3). APIs RESTful au format JSON.

• Serveur-Serveur : Communication sécurisée et authentifiée entre les microservices, idéalement au sein d'un réseau privé (VPC).

• Temps Réel : Connexion WebSocket (WSS) pour les mises à jour d'occupation en direct.

7. Exigences Non-Fonctionnelles

7.1 Exigences de Sécurité

ID	Description
NFR-SEC-01 (Chiffrement)	Toutes les données en transit doivent utiliser TLS 1.3. Toutes les données sensibles au repos, y compris les documents téléchargés et les informations personnelles (PII), doivent être chiffrées avec AES-256.
NFR-SEC-02 (Authentification & Autorisation)	Authentification par JWT avec MFA pour tous les rôles admin. Un RBAC strict doit être appliqué au niveau de la Passerelle API. Les mots de passe doivent être hachés avec Argon2id.
NFR-SEC-03 (Sécurité des QR)	Les jetons QR doivent être signés en utilisant la cryptographie asymétrique (RSA-2048 ou ECC) et inclure un horodatage d'expiration.
NFR-SEC-04 (Test d'Intrusion)	Le système doit subir un test d'intrusion complet par un tiers accrédité avant le lancement en production.

7.2 Exigences de Conformité

ID	Description
NFR-COMP-01 (Alignement ISO 27001)	La conception du système et les procédures opérationnelles doivent s'aligner sur les contrôles de la norme ISO 27001, en particulier A.9 (Contrôle d'accès), A.10 (Cryptographie), A.12 (Sécurité de l'exploitation), A.16 (Gestion des incidents), et A.18 (Conformité).
NFR-COMP-02 (Protection des Données)	Le système doit être conforme à la Loi Algérienne 18-07, garantissant le consentement de l'utilisateur pour le traitement des données et fournissant des mécanismes pour la portabilité et la suppression des données ("droit à l'oubli").

7.3 Exigences de Performance

ID	Description
NFR-PERF-01 (Concurrence)	Le système doit supporter un minimum de 10 000 utilisateurs simultanés pendant les heures de pointe sans dégradation significative des performances.
NFR-PERF-02 (Temps de Réponse)	o 95% des requêtes API doivent s'exécuter en moins de 300 ms. o Le temps de réponse pour la validation d'un QR code doit être ≤ 2 secondes (objectif < 1 seconde). o La confirmation d'une réservation doit être finalisée ≤ 5 secondes après réception de la confirmation de paiement.
NFR-PERF-03 (Mise à Jour Temps Réel)	Les mises à jour de statut d'occupation doivent être reflétées sur l'interface utilisateur dans les 3 secondes suivant l'événement déclencheur (ex: un scan).

7.4 Disponibilité et Reprise Après Sinistre (PRA)

ID	Description
NFR-AVAIL-01 (Disponibilité)	Le système doit avoir un engagement de disponibilité (SLA) de 99.5%, hors fenêtres de maintenance planifiées.
NFR-AVAIL-02 (Sauvegarde)	Des sauvegardes chiffrées complètes de la base de données doivent être effectuées quotidiennement et stockées dans un emplacement géographiquement distinct.
NFR-AVAIL-03 (Reprise Après Sinistre)	Un plan de reprise après sinistre documenté doit être en place, avec un Objectif de Temps de Reprise (RTO) de ≤ 4 heures et un Objectif de Point de Reprise (RPO) de ≤ 24 heures.

7.5 Exigences de Montée en Charge (Scalabilité)

ID	Description
NFR-SCALE-01 (Scalabilité Horizontale)	Tous les services applicatifs sans état (API, passerelles) doivent supporter la scalabilité horizontale derrière un équilibreur de charge.
NFR-SCALE-02 (Cache)	Redis doit être utilisé pour mettre en cache les données d'occupation en temps réel et les requêtes de lecture fréquentes afin de réduire la charge sur la base de données.

7.6 Auditabilité et Journalisation

ID	Description
NFR-AUDIT-01 (Journalisation Complète)	Toutes les actions définies dans la section 4.10 doivent être journalisées dans une piste d'audit immuable.
NFR-AUDIT-02 (Rétention des Journaux)	Les journaux d'audit doivent être conservés pendant un minimum d'un (1) an et doivent être facilement consultables/exportables pour les Super Admins.
NFR-AUDIT-03 (Intégration SIEM)	Le système de journalisation d'audit doit être capable de transférer les journaux vers un système SIEM (Security Information and Event Management) pour une surveillance et des alertes avancées.

7.7 Exigences d'Utilisabilité

ID	Description
NFR-UI-01 (Utilisabilité Mobile)	L'application mobile doit atteindre un score d'au moins 80 sur l'échelle d'utilisabilité système (SUS) lors des tests utilisateurs.
NFR-UI-02 (Accessibilité)	Le panneau d'administration web devrait, dans la mesure du possible, être conforme aux critères WCAG 2.1 de niveau AA.

8. Annexes

8.1 Glossaire des Termes

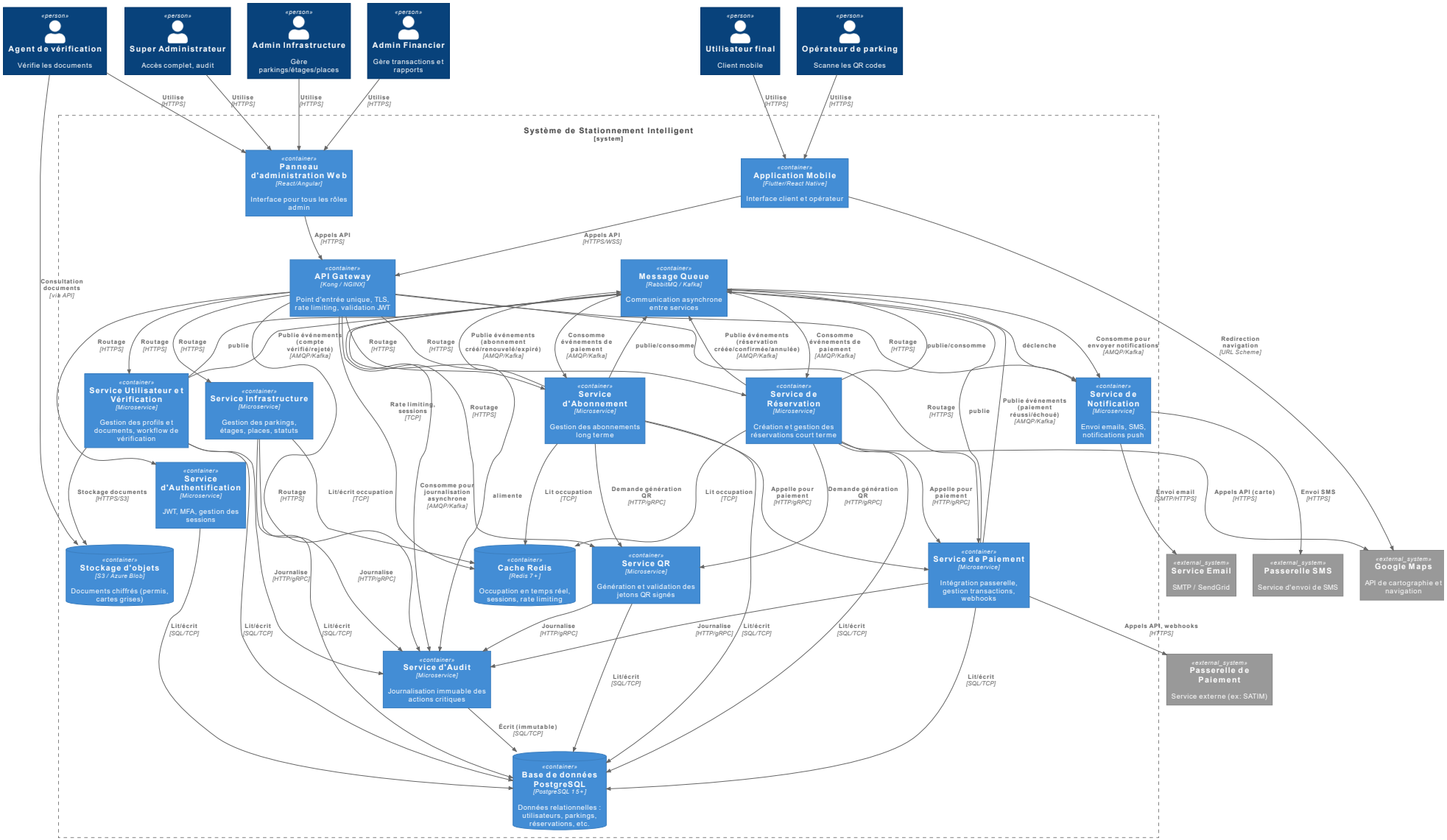
Terme	Définition
Réservation (Booking)	Une réservation de courte durée, limitée dans le temps, pour une place de stationnement.
Abonnement (Subscription)	Une assignation à long terme (mensuelle/annuelle) d'une place de stationnement spécifique à un utilisateur.
Statut de la Place	L'état actuel d'une place de parking (ex: Disponible, Réservée, Occupée).
Jeton de Validation	Les données chiffrées et signées intégrées dans un code QR.
RBAC (Contrôle d'Accès Basé sur les Rôles)	Un paradigme de sécurité qui restreint l'accès au système aux utilisateurs autorisés en fonction de leurs rôles assignés.

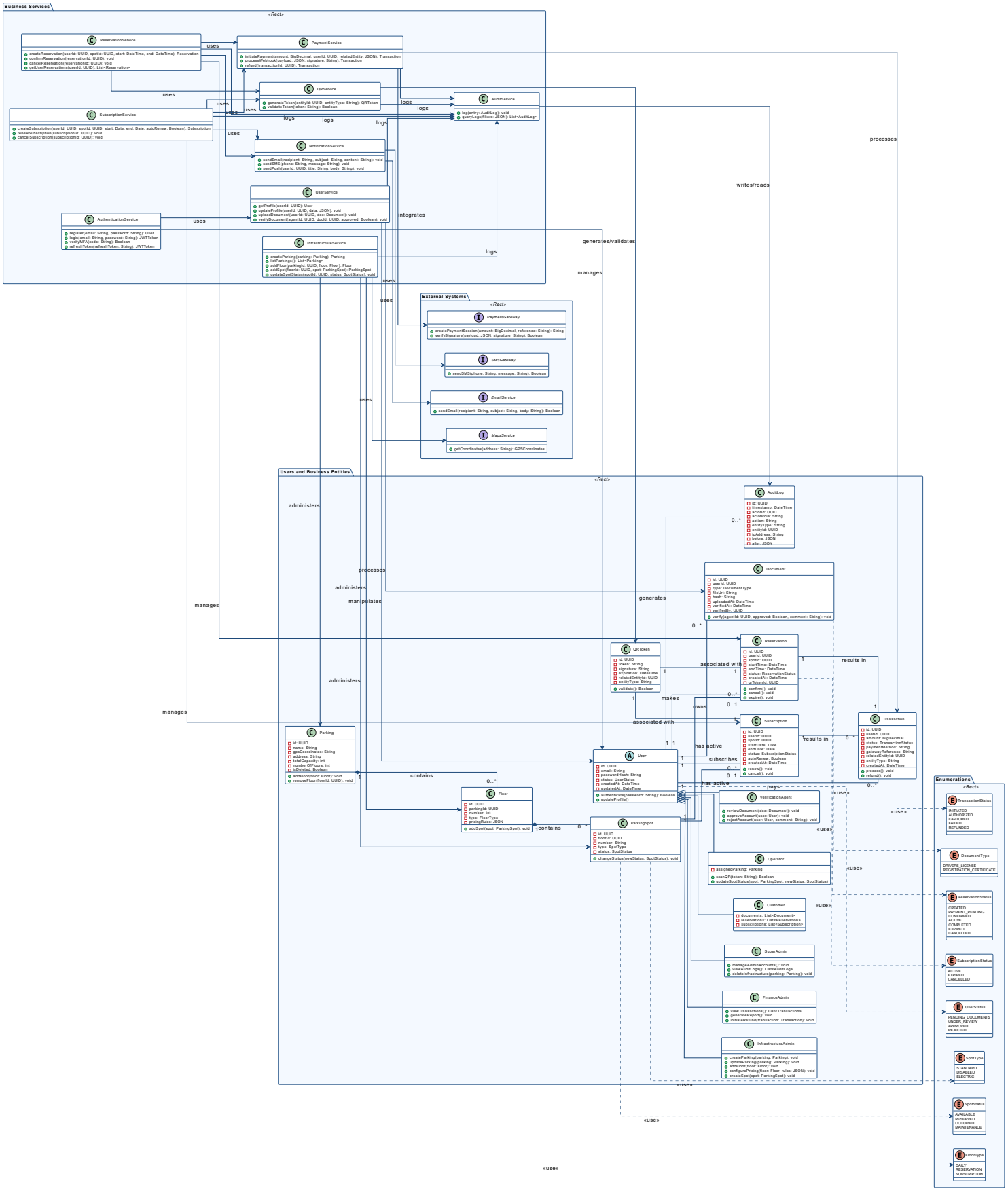
8.2 Registre des Risques et Stratégies d'Atténuation

Catégorie de Risque	Description du Risque	Probabilité	Impact	Stratégie d'Atténuation
Technique	Indisponibilité de la passerelle de paiement.	Moyenne	Élevé	Implémenter un "circuit breaker" ; maintenir une communication claire avec le fournisseur ; avoir une procédure manuelle de secours pour les cas critiques.
Sécurité	Duplication ou falsification de QR code.	Faible	Élevé	Utiliser des jetons signés et à durée limitée ; valider chaque scan côté serveur ; implémenter une limitation de débit sur les tentatives de validation.
Opérationnel	Fraude documentaire par les utilisateurs.	Moyenne	Moyen	Formation des Agents de Vérification ; mettre en place un processus de signalement des documents suspects ; stocker les hash des documents pour référence future.
Opérationnel	Abus de privilèges par un opérateur.	Moyenne	Moyen	RBAC strict ; journalisation exhaustive de toutes les actions des opérateurs ; revue régulière des journaux par le Super Admin.
Commercial	Adoption par les utilisateurs plus faible que prévue.	Moyenne	Élevé	Se concentrer sur l'expérience utilisateur (UX) ; lancer des programmes pilotes ; recueillir les retours des utilisateurs tôt et souvent.

8.3 Améliorations Futures (Hors Périmètre v1.0)

- Reconnaissance Automatique des Plaques d'Immatriculation (RAPI/ALPR) pour un accès sans barrière.
- Tarification dynamique basée sur l'IA et la demande en temps réel.
- Intégration avec des capteurs IoT pour une mise à jour automatique du statut d'occupation.
- Tableau de bord d'analyse et de reporting avancé pour l'intelligence économique.


«person»
Agent de vérification
Vérifie les documents
«person»
Super Administrateur
Accès complet, audit
«person»
Admin Infrastructure
Gère parkings/étages/places
«person»
Admin Financier
Gère transactions et rapports
«person»
Utilisateur final
Client mobile
«person»
Opérateur de parking
Scanne les QR codes
Utilise [HTTPS]
Système de Stationnement Intelligent
[system]
«container»
Panneau d'administration Web
[React/Angular]
Interface pour tous les rôles admin
«container»
Application Mobile
[Flutter/React Native]
Interface client et opérateur
Appels API [HTTPS]
Appels API [HTTPS/WSS]
Consultation documents [via API]
«container»
API Gateway
[Kong / NGINX]
Point d'entrée unique, TLS, rate limiting, validation JWT
«container»
Message Queue
[RabbitMQ / Kafka]
Communication asynchrone entre services
Routage [HTTPS]
publie
Publie événements (compte vérifié/rejeté) [AMQP/Kafka]
Publie événements (abonnement créé/renouvelé/expiré) [AMQP/Kafka]
Consomme événements de paiement [AMQP/Kafka]
publie/consomme
Publie événements (réservation créée/confirmée/annulée) [AMQP/Kafka]
déclenche
Consomme pour envoyer notifications [AMQP/Kafka]
Redirection navigation [URL Scheme]
«container»
Service Utilisateur et Vérification
[Microservice]
Gestion des profils et documents, workflow de vérification
«container»
Service Infrastructure
[Microservice]
Gestion des parkings, étages, places, statuts
«container»
Service d'Abonnement
[Microservice]
Gestion des abonnements long terme
«container»
Service de Réservation
[Microservice]
Création et gestion des réservations court terme
«container»
Service de Notification
[Microservice]
Envoi emails, SMS, notifications push
Rate limiting, sessions [TCP]
Publie événements (paiement réussi/échoué) [AMQP/Kafka]
«container»
Service d'Authentification
[Microservice]
JWT, MFA, gestion des sessions
Stockage documents [HTTPS/S3]
Lit/écrit occupation [TCP]
Consomme pour journalisation asynchrone [AMQP/Kafka]
alimente
Lit occupation [TCP]
Demande génération QR [HTTP/gRPC]
Appelle pour paiement [HTTP/gRPC]
Envoi email [SMTP/HTTPS]
Appels API (carte) [HTTPS]
Envoi SMS [HTTPS]
«container»
Stockage d'objets
[S3 / Azure Blob]
Documents chiffrés (permis, cartes grises)
«container»
Cache Redis
[Redis 7+]
Occupation en temps réel, sessions, rate limiting
«container»
Service QR
[Microservice]
Génération et validation des jetons QR signés
«container»
Service de Paiement
[Microservice]
Intégration passerelle, gestion transactions, webhooks
«external_system»
Service Email
SMTP / SendGrid
«external_system»
Passerelle SMS
Service d'envoi de SMS
«external_system»
Google Maps
API de cartographie et navigation
Journalise [HTTP/gRPC]
Lit/écrit [SQL/TCP]
Appels API, webhooks [HTTPS]
«container»
Service d'Audit
[Microservice]
Journalisation immuable des actions critiques
«external_system»
Passerelle de Paiement
Service externe (ex: SATIM)
Écrit (immutable) [SQL/TCP]
«container»
Base de données PostgreSQL
[PostgreSQL 15+]
Données relationnelles : utilisateurs, parkings, réservations, etc.


Business Services
«Rect»
ReservationService
createReservation(userId: UUID, spotId: UUID, start: DateTime, end: DateTime): Reservation
confirmReservation(reservationId: UUID): void
cancelReservation(reservationId: UUID): void
getUserReservations(userId: UUID): List<Reservation>
PaymentService
initiatePayment(amount: BigDecimal, userId: UUID, relatedEntity: JSON): Transaction
processWebhook(payload: JSON, signature: String): Transaction
refund(transactionId: UUID): Transaction
QRService
generateToken(entityId: UUID, entityType: String): QRToken
validateToken(token: String): Boolean
AuditService
log(entry: AuditLog): void
queryLogs(filters: JSON): List<AuditLog>
SubscriptionService
createSubscription(userId: UUID, spotId: UUID, start: Date, end: Date, autoRenew: Boolean): Subscription
renewSubscription(subscriptionId: UUID): void
cancelSubscription(subscriptionId: UUID): void
NotificationService
sendEmail(recipient: String, subject: String, content: String): void
sendSMS(phone: String, message: String): void
sendPush(userId: UUID, title: String, body: String): void
UserService
getProfile(userId: UUID): User
updateProfile(userId: UUID, data: JSON): void
storeDocument(userId: UUID, doc: Document): void
verifyDocument(agentId: UUID, docId: UUID, approved: Boolean): void
AuthenticationService
register(email: String, password: String): User
login(email: String, password: String): JWTToken
verifyMFACode(code: String): Boolean
refreshToken(refreshToken: String): JWTToken
InfrastructureService
createParking(parking: Parking): Parking
listParkings(): List<Parking>
addFloor(parkingId: UUID, floor: Floor): Floor
addSpot(floorId: UUID, spot: ParkingSpot): ParkingSpot
updateSpotStatus(spotId: UUID, status: SpotStatus): void
External Systems
«Rect»
PaymentGateway
createPaymentSession(amount: BigDecimal, reference: String): String
verifySignature(payload: JSON, signature: String): Boolean
SMSGateway
sendSMS(phone: String, message: String): Boolean
EmailService
sendEmail(recipient: String, subject: String, body: String): Boolean
MapsService
getCoordinates(address: String): GPSCoordinates
Users and Business Entities
«Rect»
AuditLog
id: UUID
timestamp: DateTime
actorId: UUID
actorRole: String
action: String
entityType: String
entityId: UUID
ipAddress: String
before: JSON
after: JSON
Document
id: UUID
userId: UUID
type: DocumentType
fileUri: String
hash: String
uploadedAt: DateTime
verifiedAt: DateTime
verifiedBy: UUID
verify(agentId: UUID, approved: Boolean, comment: String): void
QRToken
id: UUID
token: String
signature: String
expiration: DateTime
relatedEntityId: UUID
entityType: String
validate(): Boolean
Reservation
id: UUID
userId: UUID
spotId: UUID
startTime: DateTime
endTime: DateTime
status: ReservationStatus
createdAt: DateTime
qrTokenId: UUID
confirm(): void
cancel(): void
expire(): void
Transaction
id: UUID
userId: UUID
amount: BigDecimal
status: TransactionStatus
paymentMethod: String
gatewayReference: String
relatedEntityId: UUID
entityType: String
createdAt: DateTime
process(): void
refund(): void
Subscription
id: UUID
userId: UUID
spotId: UUID
startDate: Date
endDate: Date
status: SubscriptionStatus
autoRenew: Boolean
createdAt: DateTime
renew(): void
cancel(): void
Parking
id: UUID
name: String
gpsCoordinates: String
address: String
totalCapacity: int
numberOfFloors: int
isDeleted: Boolean
addFloor(floor: Floor): void
removeFloor(floorId: UUID): void
Floor
id: UUID
parkingId: UUID
number: int
type: FloorType
pricingRules: JSON
addSpot(spot: ParkingSpot): void
ParkingSpot
id: UUID
floorId: UUID
number: String
type: SpotType
status: SpotStatus
changeStatus(newStatus: SpotStatus): void
User
id: UUID
email: String
passwordHash: String
status: UserStatus
createdAt: DateTime
updatedAt: DateTime
authenticate(password: String): Boolean
updateProfile()
VerificationAgent
reviewDocument(doc: Document): void
approveAccount(user: User): void
rejectAccount(user: User, comment: String): void
Operator
assignedParking: Parking
scanQR(token: String): Boolean
updateSpotStatus(spot: ParkingSpot, newStatus: SpotStatus)
Customer
documents: List<Document>
reservations: List<Reservation>
subscriptions: List<Subscription>
SuperAdmin
manageAdminAccounts(): void
viewAuditLogs(): List<AuditLog>
deleteInfrastructure(parking: Parking): void
FinanceAdmin
viewTransactions(): List<Transaction>
generateReport(): void
initiateRefund(transaction: Transaction): void
InfrastructureAdmin
createParking(parking: Parking): void
updateParking(parking: Parking): void
addFloor(floor: Floor): void
configurePricing(floor: Floor, rules: JSON): void
createSpot(spot: ParkingSpot): void
Enumerations
«Rect»
TransactionStatus
INITIATED
AUTHORIZED
COMPLETED
FAILED
REFUNDED
DocumentType
DRIVERS_LICENSE
REGISTRATION_CERTIFICATE
ReservationStatus
CREATED
PAYMENT_PENDING
CONFIRMED
ACTIVE
COMPLETED
EXPIRED
CANCELLED
SubscriptionStatus
ACTIVE
EXPIRED
CANCELLED
UserStatus
PENDING_DOCUMENTS
UNDER_REVIEW
APPROVED
REJECTED
SpotType
STANDARD
DISABLED
ELECTRIC
SpotStatus
AVAILABLE
RESERVED
OCCUPIED
MAINTENANCE
FloorType
DAILY
RESERVATION
SUBSCRIPTION
uses
logs
integrates
generates/validates
manages
writes/reads
processes
administers
manipulates
generates
associated with
results in
makes
owns
has active
subscribes
pays
contains
«use»